

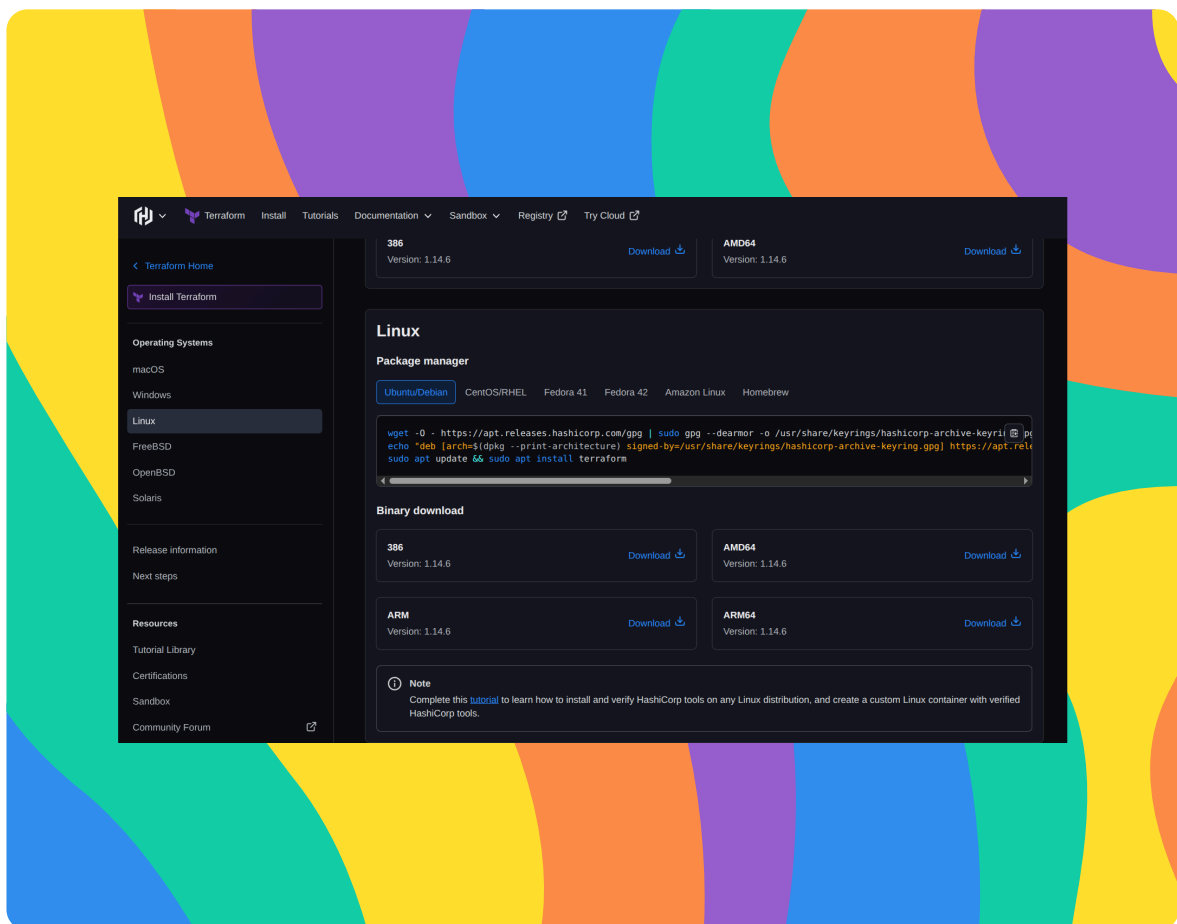


nextwork.org

Create S3 Buckets with Terraform



Ahmed Tetteh





Introducing Today's Project!

In this project, I will demonstrate the powerful effect of using an automation tool like Terraform. The goal is to install and configure Terraform. Then use Terraform to create an S3 bucket and upload files to the S3 bucket.

Tools and concepts

Services I used were Terraform, Amazon S3, and AWS CLI. Key concepts I learnt include infrastructure as code, setting up Terraform, creating and managing an S3 bucket using Terraform, uploading files to S3 with Terraform and configuring my access keys in the terminal.

Project reflection

This project took me approximately 1hr 30mins. The most challenging part was learning how to use the Terraform document to get what you want. It was most rewarding to see all the changes I made via code easily deployed within my AWS environment with Terraform.

I chose to do this project today because Terraform is one of the essential IaC tools to use within the Cloud developer space.

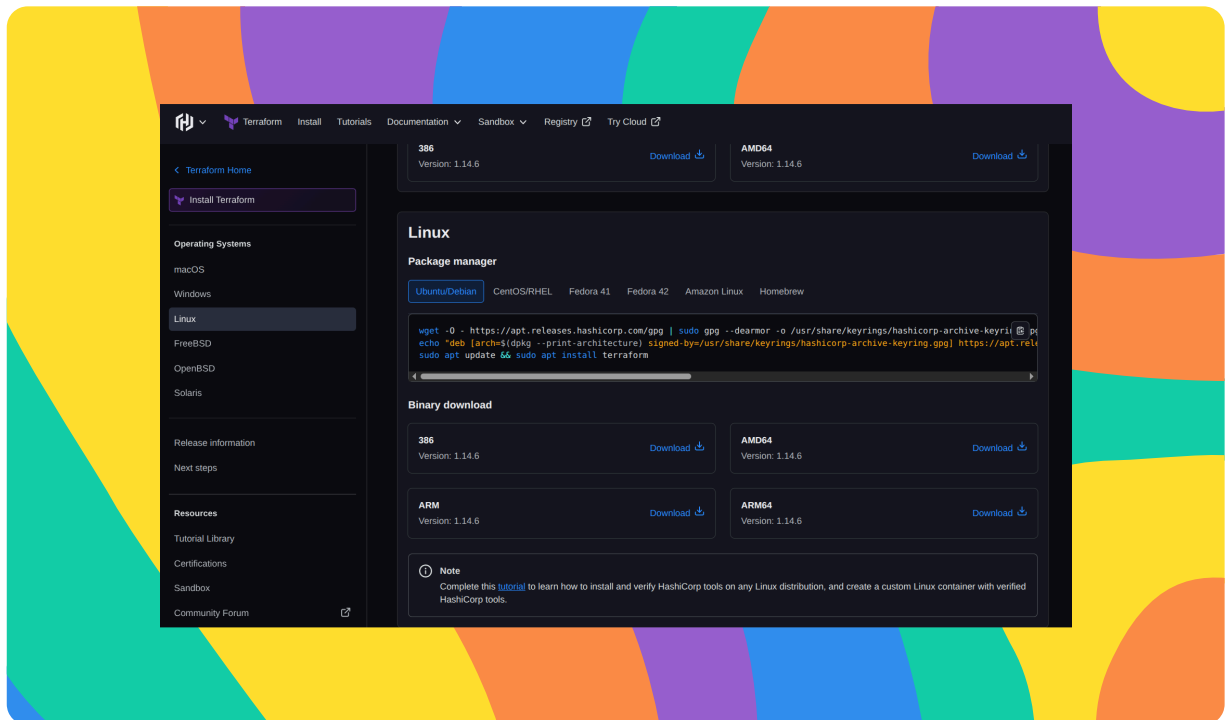


Introducing Terraform

Terraform is simply a tool to help developers build and manage their infrastructure using code.

Terraform is one of the most popular tools used for infrastructure as code (IaC), which is the practice of provisioning cloud resources such as servers, networks and storage in plain text files instead of clicking on a web console.

The `main.tf` serves as the central place or blueprint for me to define my infrastructure using Terraform's language. Terraform then uses configuration files like `main.tf` to define and manage my infrastructure.





Configuration files

The configuration is structured in multiple blocks of code. The advantage of doing this is that your code becomes more legible, and easier to modify or tweak, without it affecting other parts.

My `main.tf` configuration has three blocks

The first block indicates the Cloud Provider Terraform should work with, and in which region. The second block tells the AWS resource to provision (an S3 bucket). The third block controls who can access my S3 bucket. In this case, it's preventing all public access.



```
main.tf x
NEXTWORK_TERRAFORM
main.tf
1 provider "aws" {
2   region = "ap-northeast-2"
3 }
4
5 resource "aws_s3_bucket" "my_bucket" {
6   bucket = "nextwork-unique-bucket-fap-5638489349" # Make sure this bucket name is globally unique by typing a long random number
7 }
8
9 resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
10  bucket = aws_s3_bucket.my_bucket.id
11
12  block_public_acls       = true
13  ignore_public_acls     = true
14  block_public_policy     = true
15  restrict_public_buckets = true
16 }
17
18 Ctrl+L to chat, Ctrl+K to generate
```



Customizing my S3 Bucket

For my project extension, I visited the official Terraform documentation to learn how to provision S3 buckets in AWS using the Terraform syntax. The documentation shows all the various ways we can provision or manage cloud resources from any of the Cloud Providers (AWS, GCP or Microsoft Azure).

I chose to customise my bucket by adding a block of code that instructs Terraform to change the object ownership to the Bucket owner, regardless of who has access to the bucket and uploads an object to it. When I launch my bucket, I can verify my customization by checking the object owner properties



```
main.tf
1 provider "aws" {
2   region = "ap-northeast-2"
3 }
4
5 resource "aws_s3_bucket" "my_bucket" {
6   bucket = "nextwork-unique-bucket-fap-5638489349" # Make sure this bucket name is globally unique by typing a long random number
7 }
8
9 resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
10  bucket = aws_s3_bucket.my_bucket.id
11
12  block_public_acls       = true
13  ignore_public_acls     = true
14  block_public_policy     = true
15  restrict_public_buckets = true
16 }
17
18 resource "aws_s3_bucket_ownership_controls" "my_bucket" {
19  bucket = aws_s3_bucket.my_bucket.id
20
21  rule {
22    object_ownership = "BucketOwnerPreferred"
23  }
24 }
```



Terraform commands

I ran 'terraform init' to set up my Terraform project by establishing a connection to the cloud provider of my choice in the main.tf file. This downloaded all the connection plugins to AWS, set up a backend to keep record of my infrastructure and commands, prepares some modules(reusable pieces of code) and finally created a lock file - for versioning purposes.

Next, I ran 'terraform plan' to create an execution plan detailing the changes Terraform will make to my infrastructure based on the configurations in my main.tf file. These plans tell what Terraform will create, manage or destroy. So, this command is essential to reviewing your infrastructure before anything is created.



```
king@king-TFG5577XG:~/Desktop/Personal Lab /DevOps/nextwork_terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.34.0...
- Installed hashicorp/aws v6.34.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

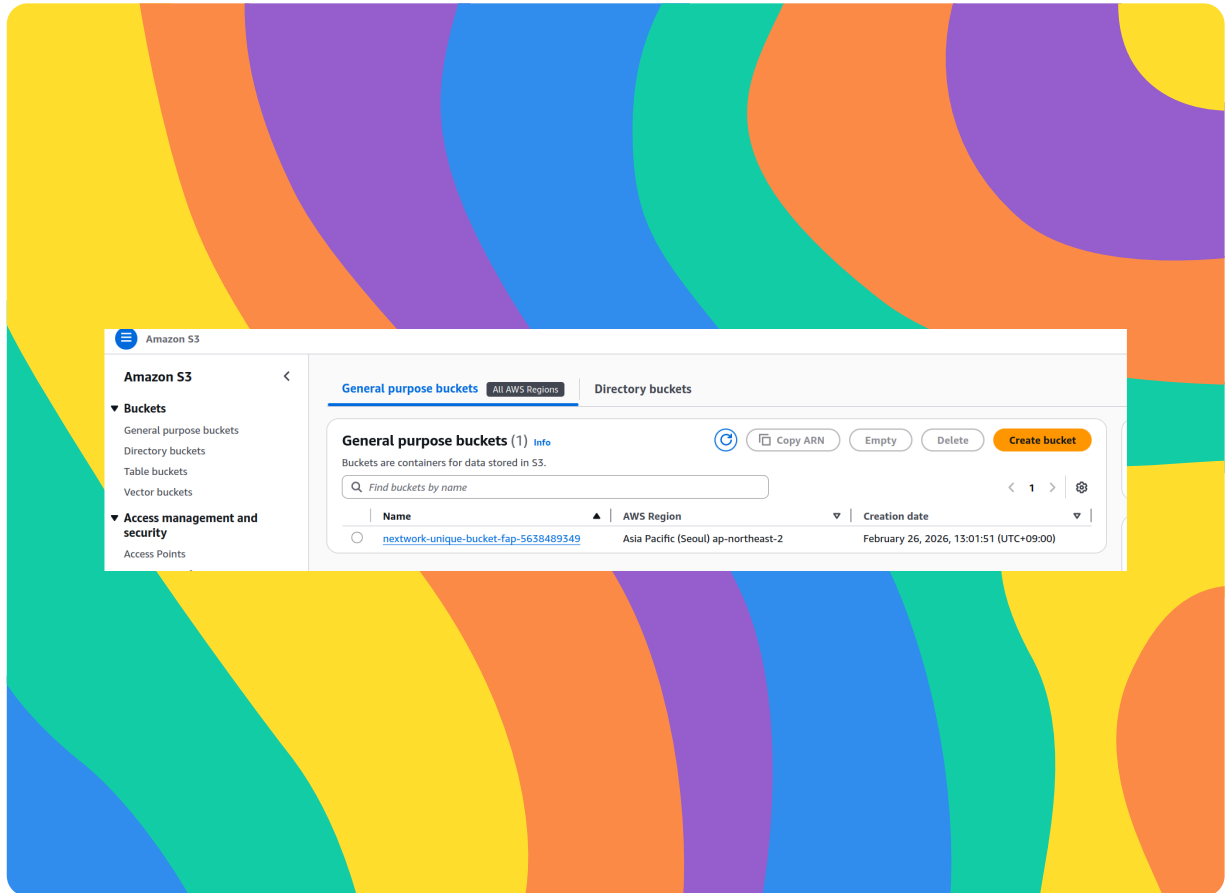
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```



Lanching the S3 Bucket

I ran 'terraform apply' to apply the changes I have made to the main.tf file. Running 'terraform apply' will affect my AWS account by either creaating or destroying resources in my account bsd on the configurations set up in the main.tf file. In my case, the file simply creates an S3 bucket.

The sequence of running terraform init, plan, and apply is crucial because you will run into an error when first trying to apply without running terraform init. Terraform firts needs to download all the necessary plugins to establish a connection to AWS and create a state file to track the state of your infrastructure. Utilizing the terrafrom plan is optional, but recommended, as it enables you to review your configuration in detail before accepting any resources to be created.





Uploading an S3 Object

I created a new resource block to upload an image into my S3 bucket.

We need to run terraform apply again because we have made changes to the configuration file. This ensures that the appropriate changes are correctly executed and made to my infrastructure.

To validate that I've updated my configuration successfully, I checked for the object in the S3 web console and downloaded the image to compare with my local one.



```
25  
26 ✓ resource "aws_s3_object" "image" {  
27     bucket = aws_s3_bucket.my_bucket.id # Reference the bucket ID  
28     key    = "image.png" # Path in the bucket  
29     source = "/home/king/Downloads/Pictures/image.png" # Local file path  
30 }  
31
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

